

Supplementary Material for: Self-Referential Anytime-Valid Byzantine-Robust Aggregation for Federated Intrusion Detection

Quang-Vinh Dang

This document contains supplementary material accompanying the main manuscript: full per-combination result and detection-statistic tables, a reproducibility checklist, a worked numerical example, a notation summary, and baseline aggregation pseudocode. Cross-references to “the main text” refer to the corresponding section of the main manuscript.

S1 Full per-combination results

Tables S1, S2, and S3 report final-round accuracy for every individual attack-type $\times$ malicious-fraction $\times$ non-IID- α combination in the robustness grid, the complete disaggregation underlying the marginal summaries in the main text’s summary table, the main text’s per-attack-type table, and the main text’s per-fraction table. We include these in full so that any specific comparison discussed in the main text can be checked directly against its source cell, and so that a reader interested in a combination we did not single out for discussion can locate it exactly.

Table S1: CICIDS2017: final-round accuracy, every attack-fraction- α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.874	0.804	0.803	0.809
ALIE	0.2	IID	0.876	0.904	0.756	0.851
ALIE	0.4	0.5	0.803	0.803	0.803	0.803
ALIE	0.4	IID	0.208	0.652	0.803	0.803
Gaussian-noise	0.2	0.5	0.803	0.803	0.803	0.656
Gaussian-noise	0.2	IID	0.803	0.803	0.803	0.738
Gaussian-noise	0.4	0.5	0.803	0.803	0.803	0.830
Gaussian-noise	0.4	IID	0.803	0.803	0.803	0.803
Label-flip	0.2	0.5	0.891	0.799	0.735	0.811
Label-flip	0.2	IID	0.864	0.825	0.846	0.904
Label-flip	0.4	0.5	0.065	0.485	0.064	0.874
Label-flip	0.4	IID	0.547	0.233	0.552	0.838

Several individual cells in Tables S1–S3 repeat an identical value across multiple aggregators for the same combination (for instance, Gaussian-noise on CICIDS2017 shows 0.803 for ARMOR-FL, FedAvg, and FoolsGold across every malicious-fraction and α combination). We confirmed this is not a data-processing artifact but reflects the underlying attack’s actual effect at the noise magnitude and malicious fractions we test: on CICIDS2017 specifically, the additive perturbation is not disruptive enough, at 20% or 40% malicious clients, to move the aggregate final-round accuracy away from what the backbone classifier converges to regardless of which of the three averaging-based rules is used, consistent with the discussion in the main text’s aggregate-results discussion. Krum, which discards rather than averages, is the only aggregator whose accuracy on these rows differs from the other three, which is itself informative:

Table S2: CICIDS2018: final-round accuracy, every attack-fraction- α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.841	0.831	0.831	0.857
ALIE	0.2	IID	0.770	0.804	0.831	0.968
ALIE	0.4	0.5	0.831	0.831	0.831	0.831
ALIE	0.4	IID	0.840	0.831	0.831	0.831
Gaussian-noise	0.2	0.5	0.831	0.831	0.831	0.886
Gaussian-noise	0.2	IID	0.831	0.831	0.831	0.917
Gaussian-noise	0.4	0.5	0.831	0.831	0.831	0.831
Gaussian-noise	0.4	IID	0.831	0.831	0.831	0.940
Label-flip	0.2	0.5	0.831	0.759	0.687	0.763
Label-flip	0.2	IID	0.871	0.945	0.397	0.936
Label-flip	0.4	0.5	0.568	0.831	0.092	0.050
Label-flip	0.4	IID	0.648	0.745	0.792	0.914

Table S3: CICIOT2023: final-round accuracy, every attack-fraction- α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.813	0.819	0.723	0.599
ALIE	0.2	IID	0.832	0.757	0.764	0.822
ALIE	0.4	0.5	0.723	0.723	0.707	0.723
ALIE	0.4	IID	0.023	0.770	0.023	0.011
Gaussian-noise	0.2	0.5	0.023	0.023	0.023	0.707
Gaussian-noise	0.2	IID	0.023	0.023	0.023	0.822
Gaussian-noise	0.4	0.5	0.023	0.023	0.023	0.595
Gaussian-noise	0.4	IID	0.023	0.023	0.023	0.823
Label-flip	0.2	0.5	0.792	0.803	0.802 ¹	0.694
Label-flip	0.2	IID	0.828	0.651	0.824	0.725
Label-flip	0.4	0.5	0.483	0.157	0.157	0.706
Label-flip	0.4	IID	0.488	0.185	0.443	0.193

the recurring identical values are a property of the averaging-based rules under this specific attack-dataset combination, not evidence that all four aggregators behave identically.

S2 Full per-combination detection statistics

Tables S4, S5, and S6 disaggregate ARMOR-FL’s own detection precision, detection recall, and benign false-exclusion rate (the main text’s evaluation-metrics section) by individual combination, underlying the marginal summaries in the main text’s detection-statistics and per-attack detection tables. A precision cell marked “–” indicates ARMOR-FL excluded no client at all during that run, making precision (a ratio over excluded clients) undefined rather than zero; we report it as undefined rather than substituting an arbitrary default, since treating an undefined ratio as zero would misleadingly imply a failed detection attempt rather than the absence of any exclusion event.

Table S4: CICIDS2017: ARMOR-FL detection statistics, every attack-fraction- α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.667	1.000	0.125
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.500
ALIE	0.4	IID	0.000	0.000	0.167
Gaussian-noise	0.2	0.5	0.000	0.000	0.125
Gaussian-noise	0.2	IID	1.000	1.000	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.500	0.500	0.125
Label-flip	0.2	IID	0.000	0.000	0.125
Label-flip	0.4	0.5	0.333	0.250	0.333
Label-flip	0.4	IID	–	0.000	0.000

Table S5: CICIDS2018: ARMOR-FL detection statistics, every attack-fraction- α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.000	0.000	0.250
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.333
ALIE	0.4	IID	0.000	0.000	0.500
Gaussian-noise	0.2	0.5	–	0.000	0.000
Gaussian-noise	0.2	IID	1.000	0.500	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.000	0.000	0.250
Label-flip	0.2	IID	0.000	0.000	0.125
Label-flip	0.4	0.5	0.333	0.250	0.333
Label-flip	0.4	IID	0.000	0.000	0.167

Recall is zero on a majority of individual combinations across all three datasets, including several where the benign false-exclusion rate is simultaneously non-zero (e.g. CICIDS2018 ALIE at malicious fraction 0.4, both α settings: recall 0.000, benign FER 0.333–0.500) – ARMOR-FL excluded only honest clients and no malicious ones at all in these specific runs. This is

Table S6: CICIOT2023: ARMOR-FL detection statistics, every attack-fraction- α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.500	1.000	0.250
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.167
ALIE	0.4	IID	0.000	0.000	0.500
Gaussian-noise	0.2	0.5	0.000	0.000	0.250
Gaussian-noise	0.2	IID	–	0.000	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.000	0.000	0.375
Label-flip	0.2	IID	–	0.000	0.000
Label-flip	0.4	0.5	0.667	0.500	0.167
Label-flip	0.4	IID	–	0.000	0.000

the most granular evidence in this paper for the honest limitation stated in the main text’s Limitations section: detection recall is not merely lower than precision in aggregate (the main text’s detection-statistics table), it is frequently exactly zero at the level of an individual run, meaning ARMOR-FL’s population-and-shift combination, at the significance levels and betting fractions of the main text’s hyperparameter table, often accumulates the twenty-five rounds of our grid without ever crossing the exclusion threshold against a truly malicious client, even while it does, in the same run, occasionally cross that threshold against an honest one.

S3 Reproducibility checklist

We report the following details explicitly to support independent reproduction, following standard reproducibility-checklist practice for empirical machine-learning papers. **Hardware.** All experiments were run on a single workstation with one NVIDIA GPU; no multi-GPU or distributed training was used, and federated clients are simulated sequentially within a single process rather than as physically separate machines. **Software.** Python 3.13 (conda environment), with the CUDA lazy-module-loading behavior explicitly disabled (`CUDA_MODULE_LOADING=EAGER`) to work around a Windows-specific crash we encountered during development. **Randomness.** Each of the 144 grid cells (three datasets $\times$ four aggregators $\times$ three attacks $\times$ two malicious fractions $\times$ two α settings, minus the cells FedAvg/Krum/FoolsGold do not require an attack-free run for) uses a single fixed random seed per configuration, not averaged over multiple seeds (the main text’s Threats to Validity section, internal validity). **Compute budget.** The full robustness grid (144 runs $\times$ 25 rounds) completes in approximately 13 hours of wall-clock time at the batch size disclosed in the main text’s hyperparameters section, following the 200-plus-hour infeasibility of the batch-size-32 configuration that motivated that disclosed deviation. **Configuration files.** Every result in this paper corresponds to a named YAML configuration file in the accompanying repository (`configs/*_robustness.yaml` and `configs/*_comparability.yaml`), and every reported number can be regenerated by re-running the corresponding configuration through the release pipeline without modification. **Data.** The three datasets are redistributed as checksummed archive chunks for reassembly rather than raw re-hosted copies, per the Data Availability declaration below, to avoid both re-download friction and any redistribution restriction on the original hosts’ full raw files.

S4 Worked numerical example

To make the ARMOR-FL algorithm in the main text concrete beyond its symbolic description, we work a single simplified round by hand for a toy federation of four active clients (rather than the six of ten used in the real grid, for brevity), of which client 4 is malicious.

Suppose the four clients' distances to the round's trimmed-mean reference point are $d = (1.0, 1.1, 0.9, 4.5)$ – clients 1–3 honest and closely clustered, client 4 an outlier. The population scale estimate is $\text{MAD}^{(t)} = \text{median}(d) = 1.05$; suppose this exceeds the floor $0.5 \cdot \text{median}(\text{recent raw MAD history})$, so no flooring correction applies this round and $\text{MAD}_{\text{eff}}^{(t)} = 1.05$. The resulting population-deviation statistics are $z = (0.952, 1.048, 0.857, 4.286)$.

Each client's e-process multiplier is $e_t = 1 + 0.5(z - 1)$ (the main text's description of the population statistic), giving $e = (0.976, 1.024, 0.929, 2.643)$. Suppose each client's population e-process entered this round at $E_{t-1} = (1.2, 0.9, 1.1, 8.5)$ (accumulated from prior rounds); the updated values are $E_t = (1.171, 0.922, 1.022, 22.474)$. With $\alpha_{\text{pop}} = 0.05$, the exclusion threshold is $1/\alpha_{\text{pop}} = 20$; only client 4 exceeds it ($22.474 \geq 20$), so clients 1–3 continue unaffected by the population gate.

Client 4's shift statistic (the main text's description of the self-shift test) compares its current $z = 4.286$ to the median of its own two-round baseline; suppose that baseline was $(0.9, 1.0)$ (client 4 looked ordinary before an attack began at this round), giving $\text{shift} = 4.286/0.95 = 4.512$, comfortably exceeding the shift threshold. Since client 4 satisfies both the population-evidence condition and the shift condition, it is hard-excluded this round: $w_4^{(t)} = 0$, and it enters probation.

For the surviving clients, the trust weight (the main text's description of the trust weight) uses $\tau = -\log(0.05) \approx 2.996$. Client 1's population e-process is $E_1 = 1.171$, giving $\log E_1 = 0.158$ and $\text{trust_scale}_1 = \text{sigmoid}(-4(0.158/2.996 - 1)) = \text{sigmoid}(3.789) \approx 0.978$: client 1 retains essentially full weight, appropriately, since its deviation is negligible relative to the exclusion threshold. Had the naive, unrescaled sigmoid been used instead ($\text{sigmoid}(-4 \times 0.158) = \text{sigmoid}(-0.632) \approx 0.347$), client 1 – an entirely honest client with only mild deviation – would have been reduced to roughly a third of its due weight, the precise failure mode described in the main text's description of the trust weight and confirmed on real data via the trust-weight trace that motivated that correction.

We now contrast this with the undetectable case discussed as the first limitation in the main text's Limitations section: a constant, moderate-magnitude attacker present from round one. Suppose client 4 is instead Byzantine from the start, with $z_4^{(t)} \approx 2.0$ (below the gross-outlier threshold of 4) on every round, including the two-round clean-baseline period (the main text's attack-implementation section). Its shift baseline, computed from its own first two rounds, is then $\text{median}(2.0, 2.0) = 2.0$, and its shift statistic in every subsequent round is $\text{shift}_4^{(t)} = 2.0/2.0 = 1.0$ – indistinguishable from a client that has never deviated at all, since the baseline itself already encodes the attacker's bias. The population e-process for this client still accumulates evidence steadily (by the main text's eventual-exclusion proposition, since $\mu = 2.0 > 1$ persistently) and will eventually cross the population threshold alone, but the shift gate of the main text's description of the self-shift test never fires, so hard exclusion never triggers under the AND condition in the ARMOR-FL algorithm in the main text; only the smooth trust-weight downweighting of the main text's description of the trust weight applies, converging to a partial but non-zero weight as $\log E_{\text{pop},4}^{(t)}$ grows past τ . This is the precise mechanical realization of the information-theoretic limit stated in the main text's Limitations section: no adjustment to b , α_{shift} , or the gross-outlier threshold recovers hard exclusion here, because the client's own baseline is contaminated by the same bias the test is trying to detect, and no signal in this paper's threat model (the main text's threat-model description) distinguishes a contaminated baseline from a genuinely elevated but honest one.

S5 Notation summary

Table S7 collects the symbols introduced across the main text’s Method section for reference, since the method combines notation from anytime-valid testing, robust statistics, and standard federated learning.

Table S7: Notation summary

Symbol	Meaning
K	Number of clients in the federation
$\mathcal{M}$	Unknown subset of malicious clients
$\mathcal{S}_t$	Clients active (selected) in round t
$\theta^{(t)}$	Global parameter vector after round t
$\theta_k^{(t)}$	Client k ’s local update in round t
$(\mathcal{F}_t)$	Filtration of aggregator information through round t
(E_t)	An e-process (the main text’s e-process definition)
α	Significance level in Ville’s inequality (the main text’s Ville’s-inequality theorem)
λ	Betting fraction in the testing-by-betting construction
μ_0	Fixed null mean for the population statistic ($\mu_0 = 1$)
$m^{(t)}$	Coordinate-wise trimmed-mean reference point, round t
$d_k^{(t)}$	Client k ’s distance to $m^{(t)}$
$\text{MAD}^{(t)}$	Round- t median-absolute-deviation scale estimate
$\text{MAD}_{\text{eff}}^{(t)}$	Floored (corrected) scale estimate
$z_k^{(t)}$	Client k ’s population-deviation statistic, $d_k^{(t)} / \text{MAD}_{\text{eff}}^{(t)}$
$E_{\text{pop},k}^{(t)}$	Client k ’s population e-process value
$\text{shift}_k^{(t)}$	Client k ’s self-shift statistic (the main text’s description of the self-shift test)
b	Shift-test baseline window length ($b = 2$)
$\alpha_{\text{pop}}, \alpha_{\text{drift}}, \alpha_{\text{shift}}$	Per-test significance levels (each 0.05)
τ	Trust-weight rescaling threshold, $-\log(\alpha_{\text{pop}})$
β	Trust-weight sigmoid sharpness ($\beta = 4$)
$w_k^{(t)}$	Client k ’s aggregation weight, round t
n_k	Client k ’s local sample count

S6 Baseline aggregation pseudocode

For direct comparison with the ARMOR-FL algorithm in the main text, Algorithms S1 and S2 give Krum and FoolsGold in the same notation, corresponding to the descriptions in the main text’s description of the self-shift test.

Algorithm S1 Krum aggregation, one round

- 1: **Input:** active client updates $\{\theta_k^{(t)}\}_{k \in \mathcal{S}_t}$, assumed Byzantine count f
- 2: **for** each active client k **do**
- 3: $D_k \leftarrow$ sum of squared distances from $\theta_k^{(t)}$ to its $|\mathcal{S}_t| - f - 2$ nearest neighbors in $\{\theta_j^{(t)}\}_{j \in \mathcal{S}_t}$
- 4: **end for**
- 5: $k^* \leftarrow \arg \min_k D_k$
- 6: **Output:** $\theta^{(t+1)} \leftarrow \theta_{k^*}^{(t)}$

Comparing Algorithm S1 against the ARMOR-FL algorithm in the main text makes concrete the mechanical explanation offered in the main text’s discussion of why Krum wins: Krum’s

Algorithm S2 FoolsGold aggregation, one round

- 1: **Input:** active client updates $\{\theta_k^{(t)}\}_{k \in S_t}$, per-client update history
 - 2: **for** each pair of active clients (i, j) **do**
 - 3: $c_{ij} \leftarrow$ cosine similarity between i 's and j 's cumulative update history
 - 4: **end for**
 - 5: **for** each active client k **do**
 - 6: $v_k \leftarrow \max_{j \neq k} c_{kj}$ (highest similarity to any other client)
 - 7: $w_k \leftarrow 1 - v_k$, rescaled and logit-transformed per the FoolsGold algorithm (see the main text)
 - 8: **end for**
 - 9: **Output:** $\theta^{(t+1)} \leftarrow$ weighted average of $\{\theta_k^{(t)}\}$ with weights $\{w_k\}$
-

output is a single client's update, verbatim, with no blending step at all, whereas ARMOR-FL (and FoolsGold, Algorithm S2) always produce a weighted combination across every active client passing the exclusion gate. The absence of any blending in Krum is precisely why a single successfully-identified honest client fully determines the round's aggregate regardless of how many other active clients are compromised, at the cost of discarding every other client's legitimate contribution outright even when most of them are honest.